

Scene Classification

Introduction

Classifying real-world scenes presents a unique computer vision challenge due to the high semantic overlap between environments (for example, distinguishing a “bedroom” from a “hotel room”). This project, developed during an internship at EICT IIT Guwahati under the guidance of Prithvijit Guha Sir, aims to build an efficient, accurate, and deployable scene classification pipeline capable of categorizing images into 314 distinct environments.

To achieve this, two contrasting architectures are evaluated: MobileNetV2, a lightweight Convolutional Neural Network, and DINOv2, a Vision Transformer. The focus of the work is to minimize Cross-Entropy Loss, maximize Top-1 and Top-5 accuracy, and analyze the trade-offs between computational cost (FLOPs/latency) and semantic understanding.

The report documents the progress from dataset preparation and model training through evaluation, computational profiling, pipeline construction, and full-stack deployment.

Tasks / Progress

- **Dataset Preparation: Auto-sorting Kaggle subset of MIT Places365**
- **Choosing Architectures: MobileNetV2 vs. DINOv2 (ViT-S/14)**
- **Deciding Evaluation Techniques: Top-1/Top-5 Accuracy, Latency, and FLOPs Analysis**
- **Training MobileNetV2: Fine-tuning the CNN backbone**
- **Training DINOv2: Training a custom MLP head on a frozen Transformer**
- **Final Model Analysis: Comparing training dynamics and efficiency**
- **Deployment: Full-stack React (Vite) and FastAPI web application**

Project Objective

The central objective is not simply to identify the most accurate model, but to quantify the accuracy–efficiency trade-off so that the final system can be deployed appropriately for real-time or server-side use.

Dataset for Evaluating the Model

To ensure our evaluation reflects complex, real-world environments, we utilized a curated Kaggle subset of the official MIT Places365 dataset. The target classification problem contains 314 scene categories.

Data Ingestion & Preprocessing

The raw dataset required significant structural modification for PyTorch compatibility. An automated Python pipeline was developed to prepare the validation images and class structure.

- Download the official MIT mapping files: categories_places365.txt and places365_val.txt.
- Map numerical class IDs to 314 human-readable folder strings.
- Sorted 36,500 validation images into their respective categorical directories.
- Apply robust training transformations including RandomResizedCrop, RandomRotation, and ColorJitter.
- Normalize tensors using standard ImageNet mean and standard deviation values.

Evaluation Techniques

The model evaluation was performed on held-out image sets using the same 314-class label mapping. The validation evaluation covered 7,298 images, while the test evaluation contained 7,302 images. MobileNetV2 and DINOv2 was evaluated on the final test set as part of the model comparison.

Dataset / Stage	Images	Purpose
Prepared validation subset	36,500	Dataset organization and preprocessing
Validation evaluation	7,298	Top-1 / Top-5 validation accuracy
Test evaluation	7,302	Final Top-1 / Top-5 test accuracy
Final model comparison	—	Accuracy, loss, FLOPs, and latency

Preprocessing for Inference

For deployment, the uploaded image is converted to RGB, resized to 256×256 pixels, center-cropped to 224×224 pixels, and normalized using ImageNet statistics. This ensures the input distribution matches the conditions expected by the trained models.

Model Architectures & Training

The project prioritizes two fundamentally different approaches to feature extraction and classification: a lightweight CNN optimized for efficiency and a self-supervised Vision Transformer optimized for rich visual representations.

1. MobileNetV2 (Lightweight CNN)

MobileNetV2 was selected as the baseline for edge-device efficiency.

- Strategy: The initial CNN backbone was frozen while the final five feature blocks (features[14:]) were explicitly unfrozen so the model could adapt its feature extraction to Places365.
- Head: A custom classifier head with Dropout (p=0.5) was attached to map the learned representation to the 314 scene classes.
- Training objective: Minimize Cross-Entropy Loss while improving validation Top-1 and Top-5 accuracy.

2. DINOv2 (Vision Transformer)

DINOv2 (dinov2_vits14 / ViT-S/14) represents a different feature-extraction paradigm, using self-supervised learning to produce robust patch-level image representations.

- Strategy: The Transformer backbone was completely frozen and used as a universal feature extractor.
- Head: A custom 2-layer MLP head (Linear → ReLU → Dropout → Linear) was trained to map DINOv2 embeddings to the 314 target classes.
- Training behavior: Because the backbone remained frozen, the classification head learned rapidly and required early stopping to limit overfitting.

Training Dynamics

MobileNetV2 followed a smoother, traditional convergence pattern and training was halted at Epoch 12 after the model reached its peak validation performance. DINOv2 triggered Early Stopping at Epoch 7; the best validation accuracy was observed earlier, at Epoch 3, while validation loss subsequently showed an upward trend.

The divergence between validation accuracy and validation loss for DINOv2 is consistent with a classifier head becoming increasingly confident in some incorrect predictions while the frozen feature extractor remains unchanged. The reported results therefore use the best-performing checkpoint rather than the final training epoch.

Training Curves

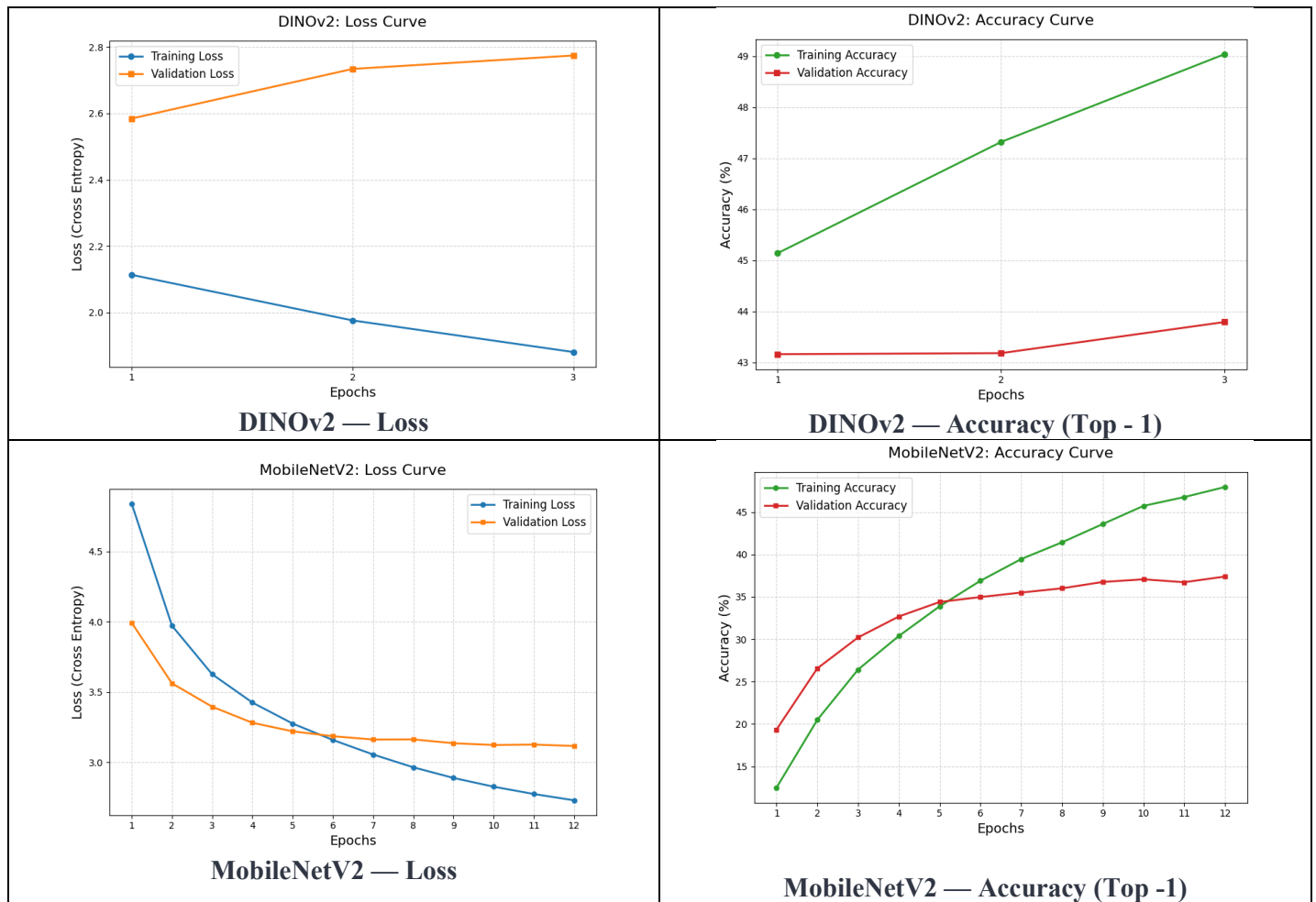


Figure interpretation: MobileNetV2 shows steady training-accuracy improvement across 12 epochs, while validation accuracy levels off around the high-30% range. DINOv2 reaches its strongest validation accuracy early, while validation loss rises as training loss falls, supporting the use of early stopping.

Evaluation Methodology

To objectively evaluate model quality without relying on a massive 314×314 confusion matrix, the project uses ranking-based classification metrics together with computational profiling.

- **Top-1 Accuracy:** The percentage of images for which the correct scene label is the model's highest-probability prediction.
- **Top-5 Accuracy:** The percentage of images for which the correct scene label appears within the model's five highest-probability predictions. This is particularly important for semantically ambiguous environments.
- **Cross-Entropy Loss:** Used during training to quantify the quality of the predicted class probability distribution.
- **Latency:** Average inference time per image after GPU warm-up.
- **FLOPs:** Theoretical computational cost required for one standard inference pass.

Training & Final Results

MobileNetV2 Results

The lightweight CNN demonstrated a smooth convergence pattern. Training was halted at Epoch 12 when the model achieved its peak validation accuracy, and the optimal weights were retained.

Metric	Training Phase	Validation Phase	Final Test Set
Loss (Cross-Entropy)	2.7287	3.1151	—
Top-1 Accuracy	47.95%	37.39%	37.72%
Top-5 Accuracy	—	—	71.25%

DINOv2 Results

DINOv2 reached its strongest validation accuracy early in training. The validation evaluation reported 43.68% Top-1 and 76.65% Top-5 accuracy on 7,298 images. On the final test set, the model achieved 43.18% Top-1 and 76.31% Top-5 accuracy across 7,302 images.

Metric	Training Phase	Validation Phase	Final Test Set
Loss (Cross-Entropy)	1.8801	2.7748	—
Top-1 Accuracy	49.04%	43.68%	43.18%
Top-5 Accuracy	—	76.65%	76.31%

Computational Efficiency (FLOPs Analysis)

A critical objective was evaluating real-world deployment feasibility. The two models present a clear trade-off between architectural complexity and inference speed.

Scenario / Metric	MobileNetV2	DINOv2 (ViT-S/14)
Parameter Count	2,626,106 (~2.62 M)	21,878,714 (~21.87 M)
Computational Cost	0.653 GFLOPs	11.050 GFLOPs
Average Latency	6.41 ms / image	7.87 ms / image

Detailed DINOv2 Profiling

Profile Item	Measured Value
Total parameters	21,878,714
Total model compute	11.050 GFLOPs
MLP head parameters	358,202
MLP head compute	0.000715 GFLOPs (0.71 MFLOPs)
Average inference latency	7.87 ms / image

Analysis

DINOv2 achieves substantially stronger semantic ranking performance, with 76.31% Top-5 test accuracy compared with 71.25% for MobileNetV2. However, DINOv2 also requires approximately 16.9× more theoretical compute per image (11.050 vs. 0.653 GFLOPs) and has a larger parameter footprint. MobileNetV2 therefore provides the stronger efficiency profile for real-time or resource-constrained edge deployment, whereas DINOv2 is better suited to server-side inference where richer scene representations are prioritized.

Model Comparison

Model	Top-1 Test	Top-5 Test
MobileNetV2	37.72%	71.25%
DINOv2 (ViT-S/14)	43.18%	76.31%

Pipeline Architecture

The final system is organized as a dual-model inference pipeline. A local image uploaded through the React frontend is sent to a FastAPI endpoint, preprocessed using the same PyTorch-compatible transformations used during model development, and then passed through both trained classifiers.

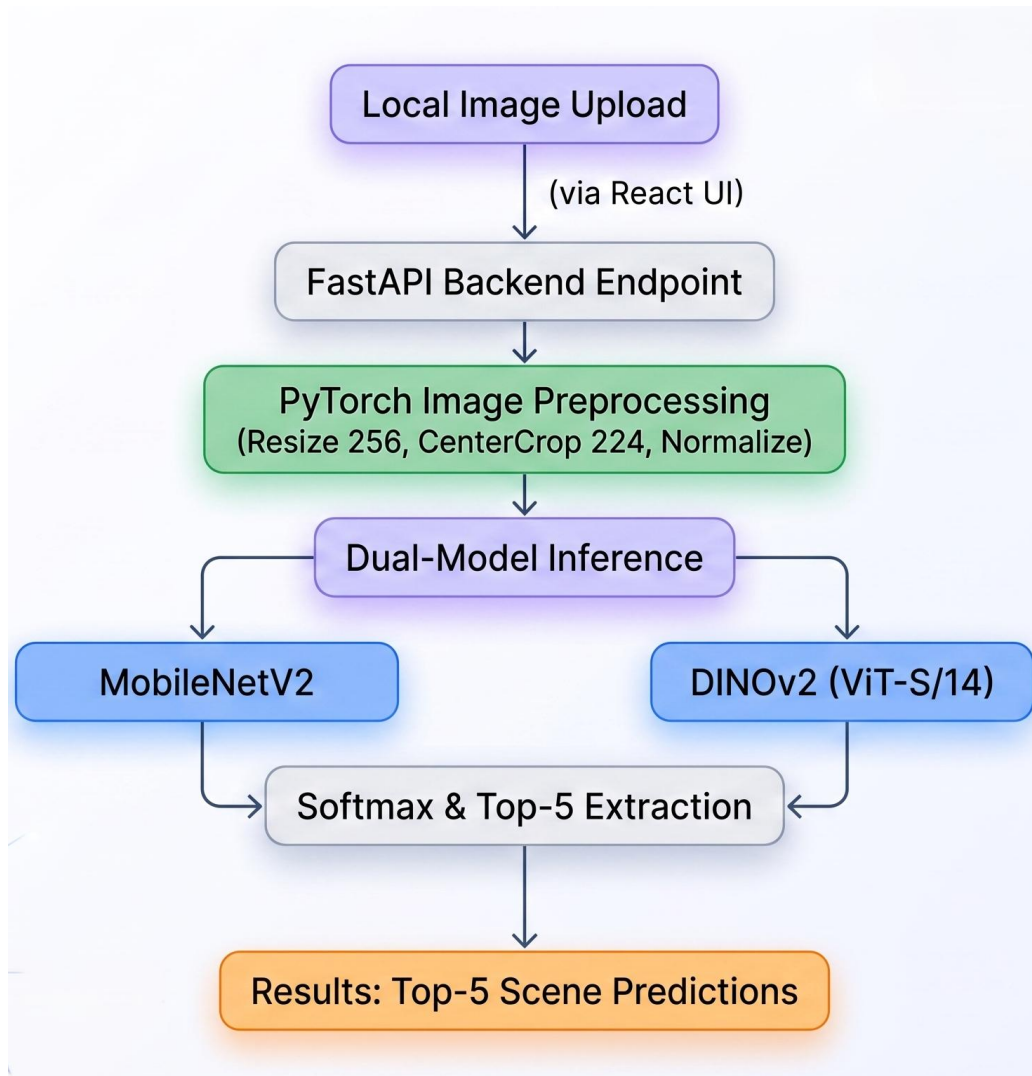


Figure 1. Final dual-model scene classification pipeline.

1. Data Ingestion & Preprocessing

The pipeline is triggered when a user uploads an image via the React frontend. The file is sent directly to the FastAPI backend where it undergoes strict normalization to match the models' training conditions.

- **Tensor Transformation:** The image is converted to RGB, resized to 256×256, and center-cropped to 224×224 pixels.
- **Normalization:** The tensor is normalized using standard ImageNet mean and standard deviation values.

2. Dual-Model Inference Engine

- **MobileNetV2 Pipeline:** Processes the tensor through the fine-tuned lightweight CNN backbone optimized for fast feature extraction.
- **DINOv2 Pipeline:** Processes the tensor through a completely frozen Vision Transformer and its custom MLP classification head.

3. Confidence Scoring & Output Formatting

- The raw logits from both models are independently passed through Softmax to calculate probability distributions.
- `torch.topk` is used to filter the 314 possible classes down to the five most confident predictions for each model.
- The structured prediction data is returned to the frontend as a JSON payload for side-by-side comparison.

Pipeline Architecture

Image Upload → FastAPI

The React UI provides the user-facing image upload interface. Once an image is selected, it is transferred to the FastAPI backend endpoint responsible for inference. Keeping model execution on the backend allows both PyTorch models to remain loaded in GPU memory and avoids transferring model weights to the client.

FastAPI → PyTorch Preprocessing

The backend converts the incoming image into a consistent RGB tensor and applies the deployment transformation sequence: Resize(256), CenterCrop(224), and ImageNet normalization. This stage ensures that the image entering both models has the expected spatial dimensions and numerical distribution.

Preprocessing → Dual-Model Inference

The normalized tensor is evaluated by MobileNetV2 and DINOv2. MobileNetV2 uses its partially fine-tuned CNN feature extractor and classifier, while DINOv2 uses its frozen ViT-S/14 representation followed by the trained MLP head.

Inference → Top-5 Predictions

Each model produces 314 logits, one for each target environment. Softmax converts these logits into probabilities, after which torch.topk selects the five highest-scoring classes. This makes it possible to compare not only the single best prediction but also the broader semantic ranking produced by each architecture.

Web App Deployment

To demonstrate the findings interactively, the trained models were deployed into a full-stack web application featuring a side-by-side comparison dashboard.

1. Backend (FastAPI)

- **Model Initialization:** On server startup, both PyTorch .pth weights are loaded into VRAM through the FastAPI lifespan context, preventing model loading overhead during individual requests.
- **Inference Endpoint:** The backend exposes a prediction endpoint that receives the uploaded image, performs preprocessing, executes both models, and calculates Top-5 probabilities.
- **Sequential Dual Inference:** Both architectures are evaluated on the same preprocessed tensor so their outputs can be compared directly.

2. Frontend (React & Vite)

- **Interface:** A responsive React UI allows a user to upload a local image and view the model outputs.
- **API Integration:** The frontend consumes the FastAPI response and dynamically maps Top-5 predictions and confidence scores onto comparison cards.
- **Visualization:** The side-by-side presentation makes the accuracy and semantic-ranking differences between MobileNetV2 and DINOv2 immediately visible.

Deployment Objective

The deployment stage converts the experimental model comparison into an interactive demonstration. It also establishes the software structure required for future improvements such as batch inference, model selection, confidence visualization, and deployment on a dedicated GPU server.

Final Analysis

The experiments show a clear architectural trade-off. DINOv2 provides stronger scene-level semantic performance, achieving 43.18% Top-1 and 76.31% Top-5 accuracy on the final test set. MobileNetV2 achieves 37.72% Top-1 and 71.25% Top-5 accuracy, but does so with only 0.653 GFLOPs and 2.63 million parameters.

The gap is especially meaningful for ambiguous scenes, where Top-5 accuracy provides a more informative view of whether the model understands the general semantic neighborhood of the image. DINOv2's 76.31% Top-5 score indicates that its learned representation is more capable of placing the correct environment among the most plausible alternatives.

At the same time, DINOv2's 11.050 GFLOPs and approximately 21.88 million parameters make it considerably heavier than MobileNetV2. The measured latency difference is comparatively modest in the tested GPU environment, but the theoretical compute and memory footprint remain important considerations for edge deployment.